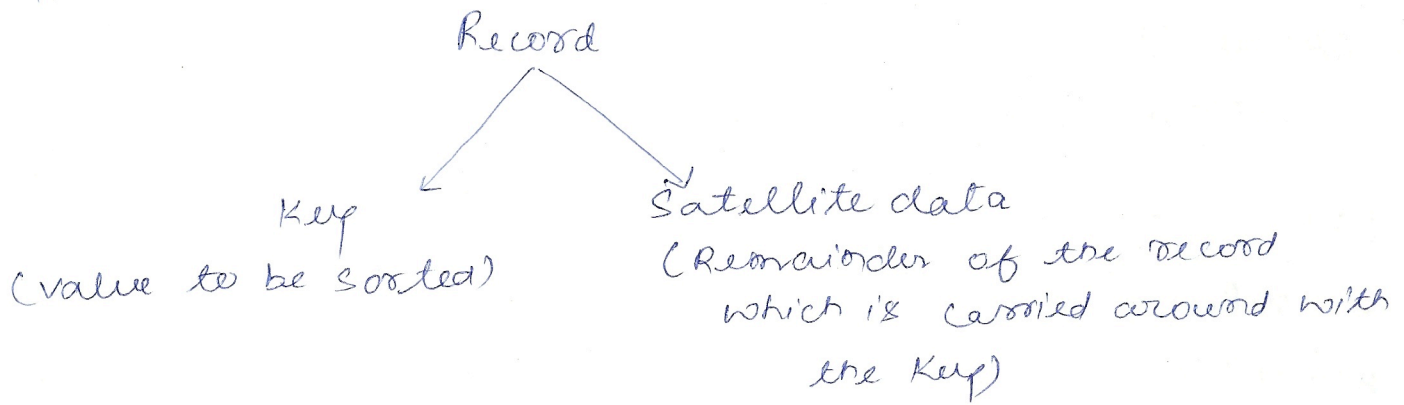
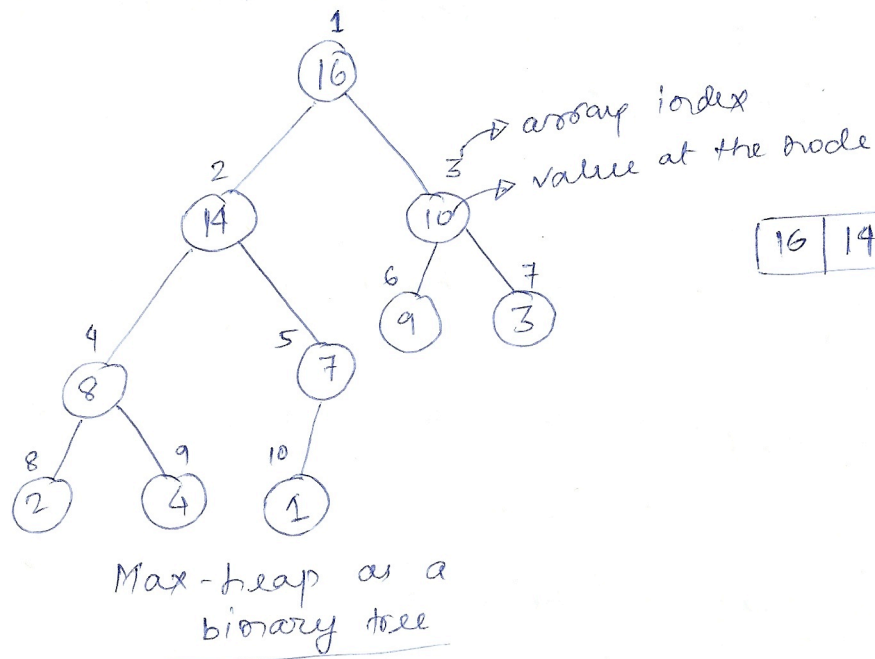


- ① → The numbers to be sorted are rarely isolated values. Usually, each is part of a record.



- ② Heapsort combines the better attributes of insertion sort and mergesort. Like insertion sort, it sorts in place: only a constant no. of array elements are stored outside the VP array at any time. Like merge sort, heapsort's running time is $O(n \log n)$.

- ③ Heap
The heap data structure is an array that can be viewed as a nearly complete binary tree.



16	14	10	8	7	9	3	2	4	1
----	----	----	---	---	---	---	---	---	---

Max-heap as an array

The tree is completely filled on all levels except possibly the lowest, which is filled from the left up to a point. An array A that represents a heap is an object that has 2 attributes:

(i) $\text{length}[A]$ (ii) $\text{heap-size}[A]$, $\text{heap-size}[A] \leq \text{length}[A]$ and

no element past $A[\text{heap-size}[A]]$ is an element of the heap. (2)

→ Given the index i of a node, the indices of its parent, left child, and right child can be computed:

PARENT(i)

return $\lfloor i/2 \rfloor$

LEFT(i)

return $2i$

RIGHT(i)

return $2i+1$

→ Max-heap

A max-heap is a heap in which the nodes satisfy the max-heap property: for every node i other than the root:-

$$A[i] \leq A[\text{PARENT}(i)]$$

Thus, the largest element in a max-heap is stored at the root.

→ Min-heap

The nodes of a min-heap satisfy the min-heap property. For every node i other than the root:-

$$A[i] \geq A[\text{PARENT}(i)]$$

Thus, the smallest element in a min-heap is at the root.

→ We use max-heaps for the heapsort algorithm and min-heaps to construct priority queues.

→ The height of a node in a heap is the number of edges on the longest simple downward path from the node to a leaf, and the height of a heap is the height of its root. Since a heap of n elements is based on a complete binary tree, its height is $\Theta(\lg n)$.

④ MAX-HEAPIFY: Maintaining the heap property

③

Its inputs are an array A and an index i into the array. When MAX-HEAPIFY is called it is assumed that the binary trees rooted at $\text{LEFT}(i)$ and $\text{RIGHT}(i)$ are max-heaps, but that $A[i]$ may be smaller than its children, thus violating the max-heap property. MAX-HEAPIFY lets the value at $A[i]$ "float down" so that the subtree rooted at i becomes a max-heap.

```
MAX-HEAPIFY( $A, i$ )
1  $l \leftarrow \text{LEFT}(i)$ 
2  $r \leftarrow \text{RIGHT}(i)$ 
3 if  $l \leq \text{heap-size}[A]$  and  $A[l] > A[i]$ 
4     then  $\text{largest} \leftarrow l$ 
5     else  $\text{largest} \leftarrow i$ 
6 if  $r \leq \text{heap-size}[A]$  and  $A[r] > A[\text{largest}]$ 
7     then  $\text{largest} \leftarrow r$ 
8 if  $\text{largest} \neq i$ 
9     then exchange  $A[i] \leftrightarrow A[\text{largest}]$ 
10    MAX-HEAPIFY( $A, \text{largest}$ )
```

→ For a subtree of size n , the running time of MAX-HEAPIFY is $O(\lg n) = O(h)$ where h is the height of the root node.

⑤ Building a heap

We can use the procedure MAX-HEAPIFY in a bottom-up manner to convert an array $A[1..n]$ into a max-heap. The elements $\lfloor n/2 \rfloor + 1, \lfloor n/2 \rfloor + 2, \dots, n$ are all leaves of the tree and so each is a 1-element heap to begin with.

```
BUILD-MAX-HEAP( $A$ )
1  $\text{heap-size}[A] \leftarrow \text{length}[A]$ 
2 for  $i \leftarrow \lfloor \text{length}[A]/2 \rfloor$  down to 1
3     do MAX-HEAPIFY( $A, i$ )
```


→ The runtime of BUILD-MAX-HEAP is $O(n)$.

④

⑥ The heapsort algorithm

The heapsort algorithm starts by using BUILD-MAX-HEAP to build a max-heap on the IP array $A[1..n]$ where $n = \text{length}[A]$. Since the maximum element of the array is stored at root $A[1]$, we put it into its correct final position by exchanging it with $A[n]$. We then discard node n from the heap by decrementing $\text{heap-size}[A]$.

HEAPSORT (A)

```
1 BUILD-MAX-HEAP ( $A$ )
2 for  $i \leftarrow \text{length}[A]$  downto 2
3   do exchange  $A[i] \leftrightarrow A[1]$ 
4    $\text{heap-size}[A] \leftarrow \text{heap-size}[A] - 1$ 
5   MAX-HEAPIFY(1)
```

→ Heapsort takes $O(n \lg n)$ time $\left[\begin{array}{l} \text{BUILD-MAX-heap} \rightarrow O(n) \\ \text{MAX-HEAPIFY} \rightarrow O(\lg n) \rightarrow \text{called } n \text{ times} \end{array} \right]$

⑦ Priority Queue

A priority queue is a set S of elements each with an associated value called a Key. A max-priority queue supports the following operations:

- (i) INSERT(S, x): inserts the element x into set S
- (ii) MAXIMUM(S): return the element of S with the largest Key
- (iii) EXTRACT-MAX(S): remove and return the element of S with the largest Key
- (iv) INCREASE-KEY(S, x, K): increase the value of element x 's Key to the new value K . K should be at least as large as x 's current Key.

Operations of a max-priority queue:-

HEAP-MAXIMUM(A)

1 return $A[1]$

▷ runtime is $O(1)$

HEAP-EXTRACT-MAX(A)

1 if $\text{heap-size}[A] < 1$

2 then error "heap underflow"

3 $\text{max} \leftarrow A[1]$

4 $A[1] \leftarrow A[\text{heap-size}[A]]$

5 $\text{heap-size}[A] \leftarrow \text{heap-size}[A] - 1$

6 MAX-HEAPIFY(A, 1)

7 return max

▷ runtime is $O(\lg n)$

HEAP-INCREASE-KEY(A, i, Key)

▷ increase the key of $A[i]$ to the new value

1 if $\text{Key} < A[i]$

2 then error "new Key is smaller than current Key"

3 $A[i] \leftarrow \text{Key}$

4 while $i > 1$ and $A[\text{PARENT}(i)] < A[i]$

5 do exchange $A[i] \leftrightarrow A[\text{PARENT}(i)]$

6 $i \leftarrow \text{PARENT}(i)$

▷ Because increasing the key of $A[i]$ may violate the max-heap property, the procedure traverses a path from this node to the root to find a proper place for the newly inserted Key.

▷ this procedure is $O(\lg n)$.

MAX-HEAP-INSERT(A, Key)

1 $\text{heap-size}[A] \leftarrow \text{heap-size}[A] + 1$

2 $A[\text{heap-size}[A]] \leftarrow -\infty$

3 HEAP-INCREASE-KEY(A, $\text{heap-size}[A]$, ~~Key~~ Key)

▷ runtime = $O(\lg n)$

_____ x x x x x x _____